MAXSQUASH

Player and Club Membership API Requirements

Version 1.1
Prepared by: Usama
Audience: Backend Team

> Purpose: Define the changes that must be added to the existing MaxSquash APIs. Existing player and club registration response structures must remain unchanged unless separately approved.

1. Document Overview

This document is a change request for the existing MaxSquash backend. It identifies the fields, public listing endpoints, validation rules, membership workflows, pagination requirements, and club approval behavior that must be added without replacing the existing player or club registration API responses.

1.1 Scope

- Add nullable club membership selections to the existing player registration request.
- Provide a public paginated endpoint to retrieve all registered active clubs for the player registration screen.
- Club-side review and approval or rejection of player membership requests.
- Add optional initial player/member IDs and the non-member booking policy to the existing club registration request.
- Store non-member booking permission and its allowed time window during club registration; do not create a separate booking-policy API for this requirement.
- Club member listing, addition, removal, and detailed player information.
- Apply standard pagination, filtering, validation, and errors. The public player and club listing endpoints must not require an auth token.

1.2 Recommended API Conventions

Convention	Requirement
Base path	/api/v1
Content type	application/json
Authentication	Bearer JWT for protected endpoints. GET /api/v1/players and GET /api/v1/clubs are public and require no auth token.
Identifiers	UUID preferred; integer IDs acceptable if already standardized
Dates	ISO 8601 UTC, for example 2026-07-20T09:30:00Z
Naming	snake_case JSON fields
Pagination	page and per_page query parameters
Soft deletion	Recommended for membership records and auditability

2. Core Business Flow

1. The player opens registration and retrieves the list of active registered clubs.
2. The player may select no club, one club, or multiple clubs where the player already holds membership.
3. For each selected club, the player must provide the relevant membership ID or membership number.
4. The player registration request creates the user account and a separate pending verification request for each selected club.
5. Each selected club reviews the player basic profile data and submitted membership ID.
6. Each club independently accepts or rejects its request.
7. On acceptance, the player is added to that club member/player list and the membership becomes verified.
8. On rejection, the player is not added to the club list and receives a notification.
9. Verified memberships are used for club access, court booking permissions, discounts, and future eligibility checks.

2.1 Membership Status Model

Status	Meaning	Allowed Transition
pending	Waiting for club review	approved or rejected

approved	Club verified the player and membership ID	suspended or removed
rejected	Club rejected the membership request	new request may be submitted
suspended	Temporarily disabled by club	approved or removed
removed	Membership association removed from club	new request may be submitted

3. Standard Response Contract

3.1 Success Response

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": { },
  "meta": { }
}
```

3.2 Validation or Business Error

```
{
  "success": false,
  "message": "Validation failed.",
  "error": {
    "code": "VALIDATION_ERROR",
    "details": {
      "club_memberships.0.membership_number": ["The membership number is required."]
    }
  }
}
```

3.3 Pagination

```
GET /api/v1/clubs?page=1&per_page=20&search=lahore&status=active

"meta": {
  "current_page": 1,
  "per_page": 20,
  "total": 67,
  "last_page": 4,
  "from": 1,
  "to": 20
}
```

Recommended limits: default per_page = 20, minimum = 1, maximum = 100. Collection endpoints should support search, sorting, and relevant status filters.

4. Player-Side Registration Requirements

GET	/api/v1/clubs	List registered clubs

4.1 Public Club List

Returns active registered clubs available during player registration. This is a public endpoint and must not require a Bearer token or any other authentication token.

Query Parameters

Field	Type	Required	Description
page	integer	No	Page number
per_page	integer	No	Items per page, max 100
search	string	No	Search by club name, city, or code
status	string	No	Default active
sort	string	No	name, created_at; prefix - for descending

Response Example

```
{
  "success": true,
  "message": "Clubs retrieved successfully.",
  "data": [
    {
      "id": "club_uuid",
      "name": "Central Squash Club",
      "logo_url": "https://.../club.png",
      "city": "Lahore",
      "address": "...",
      "status": "active"
    }
  ],
  "meta": { "current_page": 1, "per_page": 20, "total": 1, "last_page": 1 }
}
```

POST	/api/v1/auth/register/player	Register player

4.2 Existing Player Registration Request Update

Update the existing player registration request at auth/register/player. Add a nullable club_memberships field. Do not change or document a new registration response in this requirement. When clubs are submitted, create one independent pending verification request per selected club.

Fields to Add to the Existing Request

Add this nullable field to the existing auth/register/player request:

```
{
  "club_memberships": [
    {
      "club_id": "club_uuid_1",
      "membership_number": "CSC-10291"
    },
    {
      "club_id": "club_uuid_2",
      "membership_number": "LSC-8871"
    }
  ]
}
```

When no club is selected, send null or an empty array according to the existing backend convention.

Validation Rules

- club_memberships is nullable and may be an empty array.
- Each club_id must exist, be active, and be unique within the request.
- membership_number is required whenever club_id is provided.
- Duplicate email or phone must return HTTP 409 Conflict.
- Player account creation should succeed even when no club is selected.
- Club verification requests must be created transactionally with registration, or the operation must roll back.
- No membership should be marked approved during player self-registration.

5. Club-Side Registration Requirements

5.1 Public Player List

Method	Endpoint	Purpose
GET	/api/v1/players	List registered players for selection during club registration

This endpoint is public and must not require a Bearer token or any other authentication token. Return only fields needed for selection and identification; do not expose sensitive player information.

Query Parameters

Field	Type	Required	Description
page	integer	No	Page number
per_page	integer	No	Items per page; default 20, maximum 100
search	string	No	Search by player name, membership number where available, email, or phone subject to privacy rules
sort	string	No	name or created_at; prefix - for descending

Response requirements: use the standard paginated collection contract and include player id, full name, profile image URL, and only the minimum additional fields approved for the registration screen.

5.2 Existing Club Registration Request Update

POST	/api/v1/auth/register/club	Register club

Update the existing club registration request at auth/register/club. Add the initial player IDs and non-member court-booking policy to the same registration request. The booking policy must not be submitted or managed through a separate API for this scope. Do not change or document a new club registration response in this requirement.

Fields to Add to the Existing Request

Add these fields to the existing auth/register/club request:

{

```
  "initial_player_ids": ["player_uuid_1", "player_uuid_2"],
  "non_member_booking": {
    "allowed": true,
    "start_time": "10:00",
    "end_time": "16:00",
    "timezone": "Asia/Karachi"
  }
}
```

Both fields may be nullable where allowed by the rules below. The booking policy is part of club registration and must not be sent separately.

Business Rules

- When non_member_booking.allowed is false, start_time and end_time must be null, and non-members must not be allowed to book courts.
- When non_member_booking.allowed is true, start_time, end_time, and timezone are required in the club registration request.
- The end time must be later than the start time. Overnight windows should only be supported if explicitly required.
- initial_player_ids is nullable, must contain unique existing player IDs, and should not silently approve unverified membership.
- Club email and phone must be unique.
- Club registration status may default to pending if platform administrator approval is required.

6. Club Membership Verification APIs

GET	/api/v1/club/membership-requests	List membership requests

Returns player membership requests for the authenticated club. The response must include enough player data for verification.

Supported Filters

```
?page=1&per_page=20&status=pending&search=ali&sort=-created_at
```

Response Example

```
{
  "success": true,
  "data": [
    {
      "id": "request_uuid",
      "status": "pending",
      "membership_number": "CSC-10291",
      "submitted_at": "2026-07-20T09:30:00Z",
      "player": {
        "id": "player_uuid",
        "first_name": "Ali",
        "last_name": "Khan",
        "full_name": "Ali Khan",
        "email": "ali@example.com",
        "phone": "+923001234567",
        "profile_image_url": "https://.../ali.jpg"
```

```
    }
   }
  ],
  "meta": { "current_page": 1, "per_page": 20, "total": 8, "last_page": 1 }
}
```

PATCH	/api/v1/club/membership-requests/{request_id}/approve	Approve request

```
{
  "notes": "Membership verified against club records."
}
```

Approval must be idempotent. If already approved, return the existing approved membership rather than creating a duplicate. The player should be added to the club member list and notified.

PATCH	/api/v1/club/membership-requests/{request_id}/reject	Reject request

```
{
  "reason": "Membership number was not found in club records."
}
```

Rejection reason should be required, stored for audit purposes, and included in the player notification where appropriate.

7. Club Member Management APIs

GET	/api/v1/club/members	List club members

Returns approved club members. Player information should include membership number, name, profile image, contact details as permitted, membership status, and relevant timestamps.

Query Parameters

```
?page=1&per_page=20&search=ali&status=approved&sort=name
```

Response Example

```
{
  "success": true,
  "data": [
    {
      "membership_id": "membership_uuid",
      "membership_number": "CSC-10291",
      "status": "approved",
      "approved_at": "2026-07-20T11:00:00Z",
      "player": {
        "id": "player_uuid",
        "full_name": "Ali Khan",
        "first_name": "Ali",
        "last_name": "Khan",
        "email": "ali@example.com",
        "phone": "+923001234567",
        "profile_image_url": "https://.../ali.jpg"
      }
    }
  ],
  "meta": { "current_page": 1, "per_page": 20, "total": 54, "last_page": 3 }
```

```
}
```

POST	/api/v1/club/members	Add player to club

```
{
  "player_id": "player_uuid",
  "membership_number": "CSC-10291",
  "verification_mode": "club_confirmed"
}
```

Only authorized club roles may add members. The API must prevent duplicate active membership for the same player and club. The action must be audited.

DELETE	/api/v1/club/members/{membership_id}	Remove club member

```
{
  "reason": "Membership expired."
}
```

Recommended behavior: soft-delete or set status to removed. Existing booking history must remain intact. Future club-specific access should be revoked according to policy.

GET	/api/v1/club/members/{membership_id}	Get member details

Returns the full club membership record and player profile fields authorized for club administrators, including membership number, status history, image, and booking eligibility indicators.

8. Authorization and Security

- Use role-based authorization: player, club_admin, club_staff, and platform_admin as required.
- A club user may only access membership requests and members belonging to their own club.
- GET /api/v1/players and GET /api/v1/clubs must be accessible without an auth token. Apply rate limiting, safe field selection, and abuse protection to both public endpoints.
- Hash passwords using the framework approved password hashing algorithm.
- Validate uploaded media IDs and restrict file type and size.
- Do not expose sensitive player data beyond the minimum required for membership verification.
- Log approval, rejection, addition, removal, and booking-policy changes in an audit log.
- Use database transactions for multi-record registration and approval operations.

9. HTTP Status Codes and Error Codes

HTTP	Use	Example Error Code
200	Successful read or update	-
201	Resource created	-
204	Successful deletion with no response body	-
400	Malformed request or invalid business operation	INVALID_REQUEST
401	Missing or invalid token	UNAUTHENTICATED
403	Authenticated but not allowed	FORBIDDEN

HTTP	Use	Example Error Code
404	Resource not found	RESOURCE_NOT_FOUND
409	Duplicate or conflicting state	MEMBERSHIP_ALREADY_EXISTS
422	Validation failed	VALIDATION_ERROR
429	Rate limit exceeded	RATE_LIMIT_EXCEEDED
500	Unexpected server error	INTERNAL_SERVER_ERROR

10. Suggested Data Model

Entity	Important Fields	Notes
players	id, user_id, first_name, last_name, phone, profile_image_id	Player profile record
clubs	id, name, email, phone, city, address, status, logo_id, non_member_booking_allowed, non_member_booking_start_time, non_member_booking_end_time, timezone	Club profile and booking policy captured during club registration
club_membership_requests	id, club_id, player_id, membership_number, status, reviewed_by, reviewed_at, rejection_reason	One record per selected club
club_memberships	id, club_id, player_id, membership_number, status, approved_at, removed_at	Unique active club_id + player_id
audit_logs	actor_id, action, entity_type, entity_id, before, after, created_at	Security and operational traceability
notifications	user_id, type, title, body, data, read_at	Approval and rejection notifications

11. Acceptance Criteria

- Player can register without selecting a club.
- Player can select multiple active clubs and must provide a membership number for each.
- Each selected club receives a separate pending verification request.
- One club approval or rejection does not affect requests sent to other clubs.
- Approved players appear in the relevant club member list with complete required player details.
- Rejected players do not appear as club members and receive a rejection notification.
- Club can add and remove members with authorization and audit logging.
- The existing club registration request stores whether non-members can book and, when enabled, the permitted start time, end time, and timezone.
- GET /api/v1/players and GET /api/v1/clubs are public, require no auth token, and return consistent pagination metadata.
- Validation, duplicate handling, authorization errors, and not-found responses follow the standard contract.

12. Implementation Notes for Backend Team

> Important: Implement these items as changes to the existing MaxSquash APIs. Keep the current player and club registration responses unchanged. GET /api/v1/players and GET /api/v1/clubs must remain public and must not require an auth token. The non-member booking policy must be sent as part of the existing club registration request, not through a separate booking-policy API.

13. Initial Endpoint Summary

Method	Endpoint	Required Change / Purpose
GET	/api/v1/players	Public paginated player list for club registration; no auth token.
GET	/api/v1/clubs	Public paginated active club list for player registration; no auth token.
POST	/api/v1/auth/register/player	Add nullable club_memberships to the existing request. Keep the existing response unchanged.
POST	/api/v1/auth/register/club	Add initial_player_ids and non_member_booking to the existing request. Keep the existing response unchanged.
GET	/api/v1/club/membership-requests	List pending/processed verification requests for the authenticated club.
PATCH	/api/v1/club/membership-requests/{id}/approve	Approve a player membership request.
PATCH	/api/v1/club/membership-requests/{id}/reject	Reject a player membership request.
GET	/api/v1/club/members	List approved club members with pagination and player details.
GET	/api/v1/club/members/{id}	Get full authorized member details.
POST	/api/v1/club/members	Add a player to the club member list.
DELETE	/api/v1/club/members/{id}	Remove a player from the club member list while preserving history.